AI for Cloud - Architecture commune

Référence technique pour les cinq membres du groupe CSIG. Produit conversationnel : Kora. Le document définit une cible de réalisation ; il ne décrit pas une infrastructure déjà déployée.

Décisions validées par le groupe

Choix	Rôle commun
GitHub	Dépôt de référence, tâches, branches, pull requests et historique des décisions.
Antigravity	Environnement d’assistance au développement utilisé par les cinq membres.
Supabase + PostgreSQL	Plateforme de données retenue pour l’application.

Socle complémentaire proposé

Brique proposée	Convention cible et intérêt
React + TypeScript + Vite	Une interface Web de chat et un questionnaire, avec un contrat de données partagé.
Python + FastAPI	Un backend modulaire unique pour besoins, règles cloud, RAG et appels IA.
Supabase Auth / Storage	Réutiliser l’authentification et le stockage de fichiers de la plateforme.
Extension pgvector	Conserver les embeddings dans PostgreSQL pour la recherche documentaire. [R3]
API LLM et embeddings	Fournisseurs remplaçables, sélectionnés sur des essais communs.

Ces propositions doivent être adoptées collectivement avant de commencer les modules concernés. Elles ne sont pas présentées comme des décisions déjà prises. Ensuite, toute modification passe par une décision écrite dans GitHub.

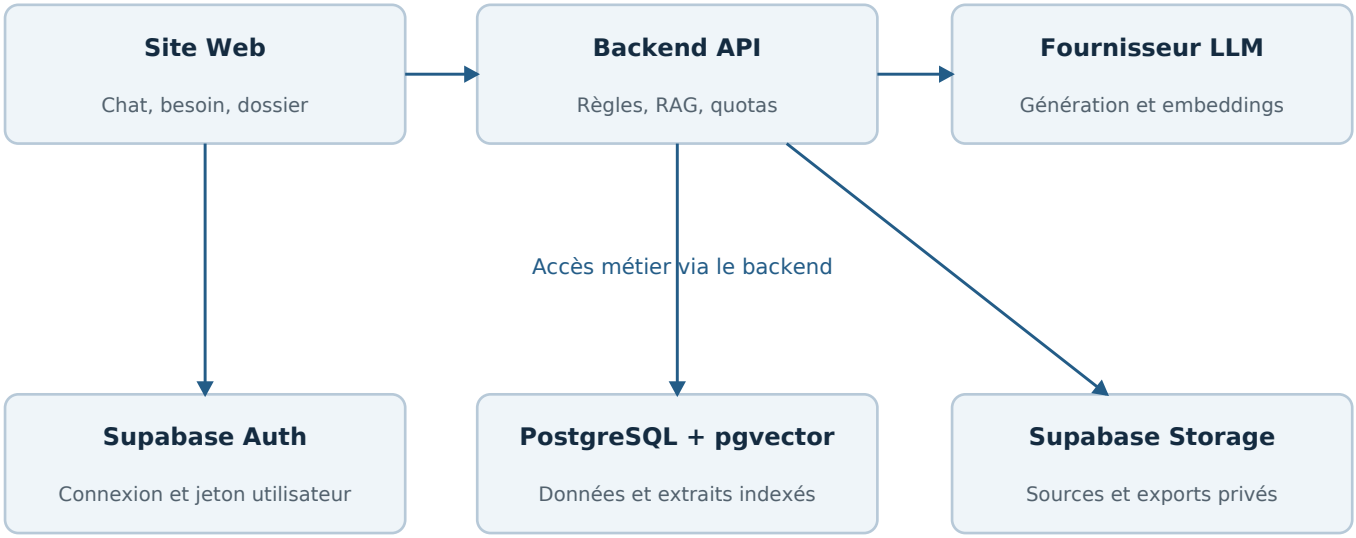
Règles structurantes

Un seul dépôt ; une seule architecture de référence ; des versions de dépendances verrouillées ; des migrations SQL partagées ; aucun secret dans Git ; aucune tâche attribuée nominativement dans ce document. Les contributions sont réalisées sur des branches distinctes puis relues.

Le cloud qui héberge l’application, le fournisseur du modèle et les trois clouds conseillés à l’utilisateur sont des choix indépendants. Kora doit comparer AWS, Azure et Google Cloud selon les besoins de l’entreprise.

Vue générale et frontières

Le schéma représente l’application cible. GitHub et Antigravity appartiennent à l’environnement de développement ; ils ne participent pas au traitement d’un message utilisateur.



Responsabilité de chaque composant

Site Web : connexion, saisie, clarification, correction du besoin, affichage des réponses et export. Il ne contient ni secret LLM ni logique privilégiée.
Backend : identité vérifiée, contrôle des accès et budgets, comparaison cloud, recherche de preuves, génération et validation du résultat.
PostgreSQL : conversations, besoins, recommandations, catalogue, extraits, versions et consommation.
Storage : fichiers de référence et dossiers exportés ; les tables conservent leur chemin et leurs métadonnées.
Services IA : calcul des embeddings et génération, avec un contexte sélectionné.

Chemin d’une demande

L’utilisateur se connecte par Auth. Le site transmet la demande au backend avec son jeton. Le backend valide la demande, vérifie les droits et réserve le budget. Il recherche des extraits, applique les critères et appelle le LLM si nécessaire. Il contrôle la réponse, enregistre le résultat et rapproche la consommation réelle du budget réservé.

Simplicité du prototype

La cible est un backend modulaire, sans imposer des microservices. L’indexation du corpus démarre par une commande dédiée ; elle n’est pas relancée à chaque message. Les traitements longs pourront nécessiter un worker, mais celui-ci ne devient une brique supplémentaire qu’après un besoin démontré.

Dépôt et environnement partagé

Le dépôt GitHub contient le code, les contrats et la documentation qui permettent aux cinq membres de reconstruire la même application.

Chemin	Contenu attendu
README.md	Objectif, prérequis, installation, lancement, tests et limites.
AGENTS.md / CONTRIBUTING.md	Consignes de développement et méthode de contribution. Fournir explicitement les consignes à l'outil si elles ne sont pas chargées automatiquement.
frontend/	Interface React/TypeScript proposée, composants et client API.
backend/app/	Routes, schémas, services métier, accès aux données et adaptateurs IA.
supabase/migrations/	Création et évolution des tables, extensions, droits, politiques RLS et fonctions SQL.
supabase/seed.sql	Données fictives reproductibles ; aucun identifiant réel.
knowledge/	Inventaire des sources et scripts d'import ; gros fichiers exclus du dépôt.
tests/ / docs/ / infra/	Cas communs, contrats API, décisions et configuration de déploiement.

Versions et commandes identiques

Retenir une version commune de Node.js, Python, PostgreSQL/Supabase CLI et un gestionnaire de dépendances par langage. Inscrire ces choix dans le README et les fichiers de configuration ; versionner les lockfiles. Les versions exactes restent à choisir lors de l'initialisation, puis doivent être identiques dans les tests automatiques et le déploiement.

Organisation locale

Chaque membre dispose de son clone, de sa branche et de ses fichiers .env locaux. Le frontend et le backend utilisent les mêmes schémas d'échange. La pile Supabase locale, via la CLI et son moteur de conteneurs requis, sert aux migrations et aux essais isolés. [R4]

Les commandes de lancement, de remise à zéro de la base locale et de tests sont documentées une seule fois. Les essais locaux doivent fonctionner avec un mode IA simulé pour éviter des appels payants à chaque modification.

Modèle de données PostgreSQL

Schéma logique proposé, à transformer en migrations. Les UUID identifient les entités ; les clés étrangères, contraintes et index portent l'intégrité des relations.

Entité	Champs / relations essentiels
auth.users / profiles	Identité gérée par Auth ; profil applicatif facultatif lié à l'UUID utilisateur.
conversations	id, owner_id, titre, état ; propriétaire lié à auth.users.
messages	id, conversation_id, rôle, contenu, état ; accès hérité de la conversation.
requirements	id, conversation_id, version, besoin JSONB validé, champs manquants.
recommendations	id, requirement_id, état, résultat validé, versions modèle/prompt/catalogue.
sources / chunks	URL, titre, fournisseur, version, empreinte ; extraits liés à une source. Embedding et identifiant du modèle associés à chaque extrait.
cloud_services / criteria	Catalogue sourcé et versionné ; règles ou paramètres de comparaison.
recommendation_sources	Lien entre recommandation, extrait exact et affirmation soutenue.
ai_requests / budget_reservations	request_id, propriétaire, opération, modèle, tokens, coût estimé, état et réservation. Écriture contrôlée côté serveur.
exports	recommendation_id, chemin privé Storage, format et état.

Contraintes à définir

Préciser NOT NULL, types, unités de coûts et unicité des requêtes. Choisir les suppressions en cascade ou contrôlées pour chaque relation. Indexer les clés étrangères et les critères de recherche utiles. Une recommandation pointe vers la version exacte du besoin ; une correction crée une nouvelle version et rend le précédent résultat identifiable comme ancien.

Les réponses structurées utilisent des schémas validés avant stockage en JSONB. Les relations et droits ne sont pas laissés à la seule logique JSON. Les extraits et références utilisés sont conservés selon la politique retenue pour expliquer un résultat même après évolution du corpus.

Identité et accès aux données

Cible proposée : authentification Supabase Auth, opérations métier via FastAPI et vérifications de propriété à chaque ressource. Les contrôles de base complètent ceux de l'application.

Accès utilisateur

Le navigateur utilise l'URL Supabase et une clé publishable pour Auth. Il transmet son access token au backend. Celui-ci vérifie signature, émetteur attendu, audience et expiration avec une bibliothèque adaptée ; décoder le contenu ne suffit pas. La vérification par JWKS suppose une signature asymétrique ; adapter la méthode au projet Supabase. [R1]

Pour les opérations effectuées au nom de l'utilisateur, le backend appelle la Data API avec la clé publishable et le JWT de cet utilisateur. Les politiques RLS limitent les lignes au propriétaire. Un client propre à la requête évite de réutiliser le jeton d'une autre session. [R2]

Politique de propriété à implémenter

Activer RLS sur les tables exposées. Conversations et besoins ne sont accessibles qu'à leur propriétaire ; messages, recommandations et exports vérifient le propriétaire de leur parent. Les écritures contrôlent aussi l'objet cible pour empêcher le rattachement à une conversation étrangère. Tester lecture, insertion, modification et suppression. [R2]

Accès internes distincts

L'ingestion, les écritures de consommation et les opérations d'administration utilisent un chemin serveur séparé et un accès privilégié limité à ces usages. Une clé secret ou service_role peut contourner RLS : la conserver uniquement sur le serveur et contrôler explicitement les droits. Un endpoint utilisateur ne doit pas devenir une passerelle arbitraire vers cet accès. [R2]

Les tables de budgets sont privées : l'utilisateur peut éventuellement consulter une vue autorisée, jamais modifier son quota. Si une fonction SQL privilégiée est utilisée, limiter ses droits d'exécution et vérifier ses entrées ; ne pas l'exposer sans contrôle.

Fichiers et cas de test

Prévoir des buckets privés, des chemins liés au propriétaire et des accès temporaires autorisés. Vérifier avec deux comptes : aucune lecture croisée, aucun export étranger et aucun accès anonyme aux données privées. Ajouter les mêmes essais sur les routes du backend.

Connaissances cloud et chaîne RAG

Le RAG est la chaîne de recherche qui fournit au modèle des passages utiles. Le prototype garde catalogue, extraits et embeddings dans PostgreSQL ; pgvector apporte la recherche de similarité. [R3]

Pipeline d’import séparé du chat

1. Sélectionner : documentation officielle sur les familles de services étudiées chez AWS, Azure et GCP.
2. Préparer : nettoyer, dédupliquer, relever URL, fournisseur, service, région pertinente et état de vérification.
3. Découper : passages cohérents avec identifiant, titre et empreinte du contenu.
4. Indexer : calculer les embeddings des extraits nouveaux ou modifiés et les stocker.
5. Contrôler : tester la recherche avec des questions et références attendues.

Recherche pendant une demande

Le backend construit une requête à partir du besoin, applique les filtres utiles et récupère un ensemble borné d’extraits. Une recherche lexicale peut compléter la similarité. Le prompt contient les consignes, le besoin validé et les passages retenus, jamais le corpus complet. Les identifiants des extraits sont associés au résultat.

Conventions indispensables

Choisir un modèle d’embeddings et une dimension uniques par index. La recherche et l’indexation emploient le même espace vectoriel. Un changement de modèle implique une réindexation ou des index séparés. Choisir un index vectoriel selon la taille observée du corpus ; mesurer sa pertinence plutôt que l’ajouter par réflexe.

Une source peut être vérifiée, à réexaminer ou retirée. Les résultats existants conservent leur traçabilité. Les documents importés sont du contexte non fiable : leurs instructions éventuelles ne doivent pas modifier les consignes du système.

Comparaison et génération

Les contraintes indispensables filtrent les options ; les préférences permettent les arbitrages. Le LLM explique les résultats en s’appuyant sur les preuves. En cas d’information insuffisante, Kora demande une précision ou signale une limite. Aucune source inventée et aucun prix non étayé ne sont acceptés comme preuve.

Contrats API et organisation du code

Contrat cible proposé sous /api/v1. Les schémas d’entrée et de sortie sont versionnés ; leur définition est commune au frontend, aux tests et à la documentation.

Route cible	Comportement
POST /conversations	Créer une conversation pour le propriétaire authentifié.
GET /conversations/{id}	Consulter uniquement une conversation autorisée.
POST /conversations/{id}/messages	Ajouter un message ; contrôler taille, droits et état.
PUT /conversations/{id}/requirements	Valider ou corriger le besoin ; créer sa version.
POST /conversations/{id}/recommendations	Lancer une génération bornée avec clé d’idempotence.
GET /recommendations/{id}	Retourner résultat, état, hypothèses et références autorisées.
POST /recommendations/{id}/exports	Créer ou récupérer un export pour le propriétaire.
DELETE /conversations/{id}	Supprimer selon la politique de conservation définie.
GET /health	État minimal du service, sans secrets ni diagnostic interne détaillé.

Modules du backend

api/ : routes et authentification. **schemas/ :** contrats et validations. **services/ :** besoins, comparaison, orchestration et export. **repositories/ :** accès Supabase. **providers/ :** adaptateurs LLM et embeddings. **core/ :** configuration, erreurs et quotas.

Résultat et erreurs

Une recommandation contient au minimum : version du besoin, fournisseurs examinés, architecture, justifications, alternatives, hypothèses, sécurité, estimation sourcée et références. Les erreurs possèdent code stable, message utile et request_id ; elles n’exposent ni clé, ni prompt système, ni trace technique.

Le streaming du texte est une option d’interface. Il ne dispense pas de valider le dossier final. Une réponse partielle ne doit pas être affichée comme une recommandation validée. Les traitements longs pourront retourner un identifiant de tâche si un worker est introduit.

Budget IA et quotas applicatifs

La base stocke les données ; les tokens sont consommés lorsque le modèle traite un contexte ou génère une réponse. Ce sont des postes différents, auxquels s’ajoutent embeddings, fichiers, hébergement et transferts.

Contrôle avant chaque appel externe

Vérifier authentification, accès à la conversation, taille des entrées et fréquence. Calculer une estimation prudente du contexte et de la sortie maximale. Réserver cette enveloppe dans un budget utilisateur et global par opération atomique ; deux requêtes concurrentes ne doivent pas dépenser le même solde. Si l’estimation dépasse le disponible, bloquer proprement avant l’appel.

Suivi et rapprochement

Enregistrer fournisseur, modèle, version tarifaire, tokens réels disponibles et coût estimé avec sa devise. Une fois la réponse obtenue, rapprocher usage et réservation. Si l’usage est inconnu après une interruption, garder un état à rapprocher au lieu de déclarer le coût nul. Une clé d’idempotence évite qu’un double clic relance la même génération ; borner les reprises automatiques.

Réduction du contexte

Conserver un historique borné et le résumé utile ; la génération d’un résumé est aussi un appel à comptabiliser. Sélectionner peu de passages pertinents et limiter leur taille. Ne recalculer les embeddings que sur les modifications. Un cache éventuel est limité au propriétaire et versionné par besoin, corpus et modèle pour éviter fuites et réponses périmées.

Limites de la maîtrise des coûts

Le suivi du backend porte les appels effectués par l’application ; il ne contrôle pas les dépenses réalisées depuis d’autres outils ou clés. Les quotas de la plateforme et les dépenses d’Antigravity sont distincts. Les alertes et le Spend Cap Supabase ne remplacent pas ce contrôle : certaines ressources, dont le calcul, ne sont pas couvertes par ce plafond. [R5]

Le groupe fixe une enveloppe approuvée avant activation de services payants. Les tests courants utilisent un fournisseur simulé ; les essais réels sont ciblés et suivis. Le modèle de génération et les seuils chiffrés restent à décider collectivement.

Configuration et secrets

Les fichiers .env.example documentent les variables. Les vraies valeurs vivent dans les fichiers locaux ignorés par Git et dans la configuration sécurisée de l'hébergement.

Variable proposée	Emplacement et sens
VITE_API_BASE_URL	Frontend : URL du backend. Valeur publique.
VITE_SUPABASE_URL	Frontend : URL du projet Auth. Valeur publique.
VITE_SUPABASE_PUBLISHABLE_KEY	Frontend : clé publique prévue pour le client ; les droits restent contrôlés côté données.
SUPABASE_URL / SUPABASE_PUBLISHABLE_KEY	Backend : projet et clé pour appels au nom de l'utilisateur.
SUPABASE_SECRET_KEY	Serveur seulement : opérations internes autorisées. Jamais préfixée VITE_.
LLM_PROVIDER / LLM_MODEL / LLM_API_KEY	Backend : fournisseur et modèle configurables ; clé secrète.
EMBEDDING_MODEL / EMBEDDING_API_KEY	Import et recherche : modèle commun ; clé secrète si requise.
ALLOWED_ORIGINS / APP_ENV	Backend : origines explicites et environnement.
MAX_CONTEXT_TOKENS / MAX_OUTPUT_TOKENS	Backend : limites choisies selon le modèle et le budget.
USER_BUDGET / GLOBAL_BUDGET	Configuration initiale ou tables privées ; unité et période définies.

Trois environnements

Local : application et Supabase local avec données fictives ; mode IA simulé par défaut.
Test partagé : projet Supabase dédié aux essais d'intégration et aux données autorisées.
Démonstration : configuration séparée, clés propres et version livrée identifiée. La séparation peut être progressive selon les accès disponibles, mais les données et secrets de démonstration ne sont pas utilisés comme bac à sable.

Droits et rotation

Chaque membre utilise son compte GitHub, Antigravity et Supabase ; pas de compte humain partagé. Les agents reçoivent les accès nécessaires à la tâche, sans accès automatique à tous les secrets. Une fuite impose révocation/rotation et correction de l'exposition, même si le fichier a ensuite été supprimé du dépôt.

Contribuer sans divergence

GitHub porte l’état partagé ; Antigravity produit des propositions à relire. Chacun peut travailler dans plusieurs briques, avec les mêmes contrats et conventions.

Cycle de contribution commun

Créer une tâche avec résultat attendu et critères. Se synchroniser sur la branche principale puis ouvrir une branche par sujet. Donner à l’agent contexte, fichiers autorisés et contrôles à effectuer. Relire les changements, tester puis ouvrir une pull request. Une autre personne vérifie le code, les effets sur les contrats, la sécurité et la documentation avant intégration.

Migrations Supabase comme référence

Les modifications des tables, droits, politiques et fonctions passent par supabase/migrations/. Tester sur une base locale réinitialisable, puis appliquer les migrations dans l’ordre à l’environnement partagé. Toute expérimentation manuelle dans le tableau de bord doit être traduite en migration avant intégration. Ne pas réécrire une migration déjà appliquée ; ajouter une migration corrective. [R4]

Contrôles automatisés proposés

Installation reproductible avec lockfiles ; compilation et contrôles TypeScript ; tests backend ; reconstruction de la base et application des migrations ; tests RLS à deux utilisateurs ; contrat API ; tests de quotas ; recherche de secrets. Les tests réels du LLM restent une exécution ciblée pour éviter des frais inutiles.

Consignes communes aux agents

Lire les documents de référence fournis, respecter le périmètre et expliquer les choix. Ne pas changer seul le framework, le modèle de données ou le fournisseur IA. Ne pas pousser de secret, modifier une base partagée sans procédure ou accepter une sortie IA sans validation. Les permissions d’Antigravity sont configurées selon l’environnement réellement utilisé. [R6]

Points d’équipe

Se retrouver fréquemment avec une démonstration visible : entrée, appel API, accès base et résultat. Consigner les décisions et blocages. Une modification structurante crée une note de décision avec contexte, options, choix, conséquences et vérifications. Le document d’architecture évolue avec les choix adoptés.

Hébergement, tests et reprise

Le fournisseur d’hébergement reste ouvert. La cible doit pouvoir accueillir un frontend statique et un backend conteneurisé sans imposer trois déploiements pour conseiller trois clouds.

Contraintes de déploiement

Choisir une région compatible avec les données et proche de la base. Activer HTTPS et les origines frontend autorisées. Configurer secrets, limites de ressources et diagnostic minimal. Prévoir l’arrêt des ressources d’essai. Cloud Run ou un autre hébergeur de conteneurs peut être évalué selon crédits, budget et besoins de traitement ; aucun déploiement payant n’est engagé par ce document.

Reprise et exploitation

Documenter installation, restauration de la base, fichiers Storage et reconstruction de l’index. La sauvegarde des tables ne doit pas être confondue avec celle des fichiers. Conserver la version du code, prompts, catalogue et corpus associée à une livraison. Vérifier la restauration sur un jeu d’essai et prévoir le retour à une version connue.

Vérification	Preuve minimale attendue
Environnement commun	Un autre membre clone, configure, reconstruit la base et lance les tests.
Isolation	Deux comptes ne lisent ni ne modifient les données l’un de l’autre.
Recommandation	Besoin corrigé, comparaison des trois clouds, justification et sources retrouvables.
Budget	Double clic et requêtes concurrentes ne contournent ni quota ni réservation.
Erreurs IA	Indisponibilité, sortie invalide et source absente sont gérées sans faux succès.
RAG	Questions connues retrouvent les passages attendus ; manque de preuve signalé.
Export et reprise	Export cohérent avec l’écran ; version et restauration vérifiables.

Les journaux contiennent request_id, état des modules, durée observée et consommation utile. Ils évitent clés, jetons et contenus sensibles inutiles. Les seuils de recette sont décidés avant l’évaluation finale ; une réussite sur le jeu de tests ne prouve pas une exactitude universelle.

Adoption de la référence commune

Le groupe dispose d'une cible cohérente, fondée sur les trois décisions déjà validées. Les compléments sont à adopter avant d'engager des développements incompatibles.

Sujet	État / décision à prendre
GitHub / Antigravity / Supabase PostgreSQL	Validés par le groupe. Centraliser dépôt, accès et configuration commune.
React/TypeScript + FastAPI	Proposition de socle ; confirmer puis verrouiller les versions.
Auth, Storage et pgvector	Extensions proposées de Supabase ; confirmer les flux et politiques.
LLM et modèle d'embeddings	Sélection à partir des mêmes cas et contraintes ; documenter les versions.
Région, plan et hébergement	Choisir selon budget, données, crédits disponibles et besoin de démonstration.
Budgets et conservation	Fixer les limites, règles de suppression et informations transmises aux services IA.

Premiers éléments à mettre en commun

README d'installation ; ce document dans docs/architecture ; conventions de contribution ; schémas API ; exemple de configuration ; première migration ; jeu fictif ; tests d'accès et mode LLM simulé. Ces éléments seront créés dans le dépôt lors de l'implémentation : ce document seul ne les installe pas.

Références officielles

[R1] Supabase - JWT et vérification

[R2] Supabase - clés API et RLS

[R2] Supabase - politiques de sécurité des lignes

[R3] Supabase - pgvector et embeddings

[R4] Supabase - développement local et migrations

[R5] Supabase - contrôle des coûts

[R6] Google Antigravity - paramètres des agents

Les tarifs, conditions d'accès et possibilités des outils évoluent. Les références officielles doivent être vérifiées lors de la configuration. Ce document spécifie une architecture cible, sans répartition nominative ni calendrier.